Electrical & Computer Project Repository Hub

Technical Documentation and Deployment Guide

Version 2.0.0 Stable Build

Domain:	Electrical and Computer project submissions
Frontend:	React, Vite, Tailwind CSS
Backend:	Node.js, Express.js
Storage:	Official GitHub project repository
Workflow:	Upload branch, pull request, auto-merge, auto-pull
Prepared By:	Vaishnav Ceh
Creator Profile:	github.com/vaishnavceh

June 11, 2026

Contents

Executive Summary

The Electrical & Computer Project Repository Hub is a full-stack web platform for collecting, organizing, and publishing student project submissions. It replaces manual Git usage for students with a guided website upload flow. The backend receives project files, validates submission data, creates a GitHub upload branch, opens a pull request, and supports an automated workflow for merging accepted uploads.

The system is designed for department-level use. It separates website source code from student project storage, avoids exposing deployment endpoints on public pages, and keeps credentials only in backend environment variables.

Current Version

This document describes **version 2.0.0 stable build**. This version keeps the current deployment workflow ready for use, with in-site PDF documentation reading, live template file browsing, official storage repository operation, safe project overwrites, and cleaner generated README files.

Version 2.0.0 Stable Update

Issues Solved

- Team number validation now accepts `team-01` through `team-100`.
- Invalid team numbers such as `team-00`, `team-101`, `team-1`, and `team-001` are rejected.
- Students can now add optional additional comments to the generated `README.md`.
- Generated README files no longer contain unfinished placeholder instructions.
- README generation now attempts form input first, readable report PDF sections second, and safe fallback text after that.
- Existing project folders can now be replaced through a new upload pull request.
- Public pages no longer display the deployed backend server address.
- The project now supports the official Electrical and Computer GitHub storage repository.
- The Auto Merge GitHub Actions workflow is documented for the storage repository.
- Templates are now live in the official repository `TEMPLATES` folder.

New Updates

- Application version updated to `2.0.0 stable build`.
- Documentation now has its own tab with a click-to-load PDF.js preview inside the website.
- Templates page now lists the actual files and folders inside the official `TEMPLATES` directory.
- Creator GitHub profile is included in project information.
- Existing project uploads can replace previous folder contents through a new pull request.
- Generated README files use form input, readable report PDF sections, and safe fallback text.
- Know More page includes solved issues and stable build update notes.
- Upload page includes a Git command window for users familiar with Git.
- Admin Pane includes frontend and backend status checks behind a passkey.
- Templates page browses the official GitHub repository `TEMPLATES` directory inside the website.
- Light and dark mode are available in the frontend interface.

Purpose and Scope

Purpose

The purpose of this website is to make student project submission structured, repeatable, and easier to maintain. Students should not need to clone repositories, create branches, or open pull requests manually. The website handles the submission flow and lets GitHub workflows manage accepted project updates.

Scope

The platform supports:

- Team-based project submissions.

- Project files and optional report PDF uploads.
- Official course, seminar, lab, presentation, and review template access.
- Optional report and working-video links.
- Optional README comments.
- GitHub pull request creation.
- Auto-merge workflow support for upload branches.
- Repository browsing inside the website.
- Admin-only status checks.

The platform does not act as a complete learning management system. Authentication, role-based student accounts, grading, and analytics are future enhancements.

Repository Strategy

The project uses two separate repositories.

Repository	Purpose
`ER_Project-Hub`	Website source code, backend source code, frontend source code, documentation, Docker setup, and deployment configuration examples.
`ELECTRICAL-AND-COMPUTER-PROJECT-REPOSITORY-OFFICIAL`	Official storage location for accepted student project files.

This separation keeps the website application clean and prevents student submissions from being mixed with source code.

System Architecture

Architecture Overview

Layer	Responsibility
Frontend	React/Vite application, student upload form, repository browser, admin pane, status UI, dark/light mode.
Backend	Express API, upload validation, file handling, GitHub branch creation, pull request creation, repository file listing.
GitHub Storage Repo	Stores accepted project files under batch, semester, subject, and team folders.
GitHub Actions	Auto-merge workflow for trusted upload branches after checks pass.
Deployment	Frontend and backend are deployed separately and connected using environment variables.

Request Flow

1. Student opens the frontend website.
2. Student fills the upload form and selects files.
3. Frontend sends form data to the configured backend service.
4. Backend validates fields, filenames, and file safety rules.
5. Backend creates an upload branch in the storage repository.
6. If the same project folder already exists, backend deletes the old project files on the upload branch.
7. Backend uploads project files and generated README content.
8. Backend opens a pull request against `main`.
9. GitHub Actions handles the auto-merge workflow for valid upload branches.
10. Accepted files become visible through the repository browser.

Technology Stack

Technology	Use

React	Builds the browser-based user interface.
Vite	Provides fast development and optimized production builds.
Tailwind CSS	Provides utility-based styling and responsive layout.
Lucide React	Provides icons for navigation, buttons, status panels, and controls.
Node.js	Runtime for the backend service.
Express.js	API routing and middleware.
Multer	Handles multipart project file uploads.
Octokit	Communicates with the GitHub REST API.
Docker Compose	Supports local development and testing.
Vercel	Intended frontend deployment platform.
Node Web Service	Intended backend deployment environment.

Frontend Pages

Home

The Home page summarizes the platform workflow. It tells users that project uploads are handled by the website, that auto-merge and auto-pull are enabled after checks pass, and that concerns should be reported to the admin. Documentation is available from its own navigation tab so the Home page does not automatically load the PDF.

Documentation

The Documentation page provides a click-to-load PDF.js preview of the latest compiled PDF documentation. The PDF is loaded only after the user requests the preview, which prevents the Home page from triggering an automatic browser download and avoids relying on blocked Google Docs embedding.

Upload Project

The Upload Project page is the main student submission page. It collects:

- Batch year, for example `batch-2027`.
- Semester, for example `semester-5`.
- Subject, for example `dbms`.
- Team number from `team-01` through `team-100`.

- Project name, for example `library-management`.
- Team members.
- Project description.
- Tools used.
- Technologies used.
- Sources or references used.
- Problem statement, optional. If omitted, the README derives it from the project description.
- Installation or setup instructions, optional.
- Steps to run or test, optional.
- Screenshots, photos, or demo output notes, optional.
- Presentation details, optional.
- Future improvements, optional.
- Additional README comments, optional.
- Project files.
- Project report PDF, optional.
- Google Drive report link, optional.
- Working video link, optional.

After upload, the page shows the created branch, pull request link, project path, and Git process window. If the upload replaces an existing project folder, the success panel reports how many existing files were replaced.

Files

The Files page lets users browse accepted project files from the official storage repository inside the website. It does not expose GitHub tokens or backend secrets.

Guidelines

The Guidelines page explains upload expectations. It clarifies that students should submit through the website and that normal uploads are handled by the automated workflow after checks pass.

Repository

The Repository page explains the required storage folder structure:

```
batches/{batch}/{semester}/{subject}/{team-number}-{project-
    name}/
```

Example:

```
batches/batch-2027/semester-5/dbms/team-01-library-management/
```

README Requirements

The README page explains what a useful project README should contain. The backend generates a starter README from form data, and students can add optional comments for special setup notes, hardware warnings, known issues, or extra context.

Hardware

The Hardware page gives guidance for electronics, electrical, embedded systems, IoT, robotics, and hardware projects. Students are encouraged to include circuit diagrams, source code, reports, simulation files, photos, and demo evidence.

Rules

The Rules page defines safe submission behavior. Students must not upload secrets, tokens, passwords, private keys, unrelated private data, or unsafe executable files.

Templates and Resources

This page shows the actual files and folders from the live official `TEMPLATES` directory in the storage repository. Students can open template folders, view template file names, open files on GitHub, and download available files before preparing their submissions.

Official templates repository:

```
https://github.com/ELECTRICAL-AND-COMPUTER-TKMCE/ELECTRICAL-AND
    -COMPUTER-PROJECT-REPOSITORY-OFFICIAL
```

Official templates folder:

```
https://github.com/ELECTRICAL-AND-COMPUTER-TKMCE/ELECTRICAL-AND
   -COMPUTER-PROJECT-REPOSITORY-OFFICIAL/tree/main/TEMPLATES
```

Know More

The Know More page shows version information, workflow status, solved issues, operational notes, and planned features. It also includes the creator GitHub profile:

```
https://github.com/vaishnavceh
```

Admin Pane

The Admin Pane is located in the lower area of the Home page. It is protected by passkeys configured through frontend environment variables.

Admin Features

- Frontend status check.
- Backend/project service status check.
- Runtime configuration check.
- Repository access check.
- Expandable command/process log.

Passkey Configuration

The frontend reads the admin passkey from:

```
VITE_ADMIN_PASSKEY=<admin-passkey>
```

Multiple passkeys can be supported with:

```
VITE_ADMIN_PASSKEYS=<passkey-one>,<passkey-two>
```

Upload Validation Rules

Required Fields

- Batch year.
- Semester.
- Subject.
- Team number.
- Project name.
- Team members.
- Project description.
- Tools used.
- Technologies used.
- Sources or references used.
- Project type.
- Confirmation checkbox.

Team Number Rule

Valid examples:

```
team-01
team-09
team-10
team-99
team-100
```

Invalid examples:

```
team-00
team-101
team-1
team-001
```

Blocked Uploads

The backend blocks filenames that appear unsafe, including executable files, private key files, environment files, token files, credential files, and path traversal attempts.

Generated README.md

For each project, the backend creates a starter `README.md`. It includes:

- Project title.
- Team members.
- Batch, semester, subject, project type, and team folder.
- Project description.
- Tools used.
- Technologies used.
- Sources or references.
- Project file location.
- Report PDF status.
- Optional report link.
- Optional working video link.
- Optional additional comments.
- `Not available.` for optional sections that are not provided and cannot be read from the report PDF.

Overwrite Behavior

When a team uploads a project to the same batch, semester, subject, team number, and project name path, the backend no longer rejects it as a duplicate. Instead, it creates an upload branch from the latest `main`, removes the existing files in that project folder on the upload branch, uploads the new files, and opens a pull request.

This means the replacement is still tracked by Git history. After the pull request is auto-merged, users can run `git pull origin main` in the storage repository to receive the overwritten project folder.

GitHub Actions Auto-Merge Workflow

The Auto Merge workflow must be created in the official storage repository, not in the website source repository.

File path:

```
.github/workflows/auto-merge-student-uploads.yml
```

Workflow:

```
name: Auto merge student uploads

on:
  pull_request:
    types: [opened, synchronize, reopened, ready_for_review]
    branches:
      - main

permissions:
  contents: write
  pull-requests: write

jobs:
  auto-merge:
    if: startsWith(github.head_ref, 'upload/')
    runs-on: ubuntu-latest

    steps:
      - name: Auto merge upload pull request
        run: gh pr merge "$PR_URL" --squash --auto --delete-
           branch
        env:
          GH_TOKEN: ${{ secrets.GITHUB_TOKEN }}
          PR_URL: ${{ github.event.pull_request.html_url }}
```

Workflow Settings

In the storage repository, configure:

- **Settings** → **Actions** → **General**.
- Enable **Read and write permissions**.

- Enable **Allow GitHub Actions to create and approve pull requests**.

The workflow only applies to branches that start with `upload/`. Normal manual pull requests are not auto-merged by this rule.

Environment Variables

Frontend

```
VITE_API_BASE_URL=<configured-project-service-url>
VITE_ADMIN_PASSKEY=<admin-passkey>
VITE_STORAGE_REPOSITORY_URL=https://github.com/ELECTRICAL-AND-
   COMPUTER-TKMCE/ELECTRICAL-AND-COMPUTER-PROJECT-REPOSITORY-
   OFFICIAL.git
```

Note: The deployed project service URL should be configured in Vercel or local environment files. It should not be hardcoded into committed source files.

Backend

```
GITHUB_OWNER=ELECTRICAL-AND-COMPUTER-TKMCE
GITHUB_REPO=ELECTRICAL-AND-COMPUTER-PROJECT-REPOSITORY-OFFICIAL
GITHUB_BASE_BRANCH=main
GITHUB_TOKEN=<server-side-token>
FRONTEND_URL=<frontend-origin>
CORS_ORIGIN=<frontend-origin>
NODE_ENV=production
PORT=10000
```

Important: The GitHub token must never be placed in frontend code, public screenshots, documentation, or Git history.

Deployment Guide

Frontend Deployment

The frontend can be deployed to Vercel.

1. Import the website source repository into Vercel.

2. Set the frontend root directory if required.
3. Add the frontend environment variables.
4. Deploy the frontend.
5. Copy the final frontend origin.
6. Update backend CORS settings with the final frontend origin.

Backend Deployment

The backend should run as a Node web service.

```
npm install
npm start
```

Backend health check path:

```
/api/health
```

Routes such as `/` or `/api` may return `Cannot GET` if not defined. That is acceptable when `/api/health` returns a valid health response.

Testing Checklist

No.	Test Item	Status
1	Frontend loads successfully.	Pending
2	Light mode and dark mode work correctly.	Pending
3	Admin Pane login works with configured passkey.	Pending
4	Backend health status check works from Admin Pane.	Pending
5	Upload form accepts `team-01` through `team-100`.	Pending
6	Upload form rejects invalid team numbers.	Pending
7	Optional README comments appear in generated `README.md`.	Pending
8	Backend creates an upload branch in the official storage repository.	Pending
9	Backend opens a pull request against `main`.	Pending
10	GitHub Actions auto-merge workflow runs for `upload/` branches.	Pending
11	Accepted project files appear in the repository browser.	Pending

Security Notes

- Store GitHub tokens only in backend environment variables.
- Never commit `.env` files.
- Rotate any token that was shown in a screenshot, browser page, terminal, or chat.
- Use the minimum GitHub permissions required for contents and pull requests.
- Prefer a department bot account or GitHub App for long-term deployment.
- Keep backend deployment endpoints out of public website pages.
- Do not allow students to submit passwords, API keys, private keys, or secret files.

Troubleshooting

Problem	Likely Fix
Frontend says backend is not reachable	Check frontend environment variable `VITE_API_BASE_URL`, backend CORS origin, and backend service status.
Upload fails with team number error	Use `team-01` through `team-100`.
Pull request is created but not merged	Check GitHub Actions permissions and confirm the branch starts with `upload/`.
Repository browser does not show files	Confirm backend environment points to the official storage repository and token has access.
Admin Pane login fails	Confirm `VITE_ADMIN_PASSKEY` or `VITE_ADMIN_PASSKEYS` is configured and redeploy frontend.

Future Enhancements

- Student login and team submission history.
- Admin dashboard for accepted uploads and rejected submissions.
- Search and filters by batch, semester, subject, team, and technology.
- More detailed GitHub Actions status inside the website.

- Course material templates, seminar templates, report templates, and lab resources.
- Department-level project analytics.

Conclusion

The Electrical & Computer Project Repository Hub provides a structured and safer workflow for collecting department project submissions. Version 2.0.0 stable build improves validation, documentation, official repository configuration, in-site PDF documentation reading, visible template directory browsing, generated README flexibility, overwrite handling, and deployment readiness. The system is prepared for frontend hosting, backend production use, and official storage repository testing.